

SYSTEMS DESIGN · MACHINE LEARNING · PUBLIC
POLICY

NexusSim

A Real-City, Real-Time Multi-Agent Traffic Simulation & Fair
Traffic-Light AI — explained from first principles, then
engineered end to end.

ENGINE

**C++ /
Quadtree /
WASM**

BRAIN

**Python
MARL /
ONNX**

GROUND TRUTH

**OpenStreetMap,
3 real cities**

OBJECTIVE

**Speed + Equity,
not just Speed**

Every architectural weak point identified during interview-style review has a concrete, engineered fix in this document.

What's Inside This Document

1. The Idea in Plain Words — Three Analogies	3
2. How the Three Parts Actually Talk to Each Other	4
3. System Architecture, End to End	5
4. Nine Problems Interviewers Would Raise — Solved	6
5. The Reward Function: Solving Credit Assignment + Fairness	9
6. The Real-City Data Pipeline	10
7. Solutions Summary Table	11
8. The 30-Second Pitch	12

The Idea in Plain Words

NexusSim is a computer program that builds a working, fake copy of a real city's traffic — and then teaches an AI to control the traffic lights so that everyone, not just people in the richest part of town, gets where they're going faster.

ANALOGY — A VIDEO GAME CITY

Imagine a SimCity-style game, but instead of a made-up map, it's built from the real roads of Ahmedabad, Chicago, or Paris. Instead of a player controlling the traffic lights by hand, an AI is playing the game — trying to get the best score, where the score is "how smoothly is traffic moving for everyone in the city, not just some of it."

The Three Building Blocks

A. The Engine (C++) — the world where the city lives

This is the part that draws every car, bus, and rickshaw moving on the map — thousands of them at once — without slowing down or freezing.

ANALOGY — A VIDEO GAME ENGINE

The same way Unity or Unreal keeps hundreds of characters moving smoothly on screen without lag, this engine keeps thousands of vehicles moving smoothly through the simulated city. C++ is used because it is the fast, no-nonsense language real game engines are actually built in.

B. The Brain (Python) — the part that learns

A machine learning model that watches the traffic and slowly learns, through trial and error, which traffic-light decisions actually reduce jams — instead of following a fixed rulebook.

ANALOGY — TEACHING A NEW EMPLOYEE BY LETTING THEM PRACTICE

Instead of writing a strict rulebook ("always do X after Y"), you let the AI run the simulation over and over. It slowly notices which choices lead to smoother traffic and which ones cause pile-ups further down the road — the same way a new employee improves through repeated practice and feedback, not by memorizing a manual.

C. The Real City Map (OpenStreetMap) — the ground truth

Instead of inventing a clean, imaginary grid city, the program pulls in real map data — actual roads, intersections, and lane counts — so the simulation reflects a real place.

ANALOGY — A REAL MAP INSTEAD OF A HAND-DRAWN ONE

It would be easy to make up a tidy, grid-shaped fake city — but real cities are messy: narrow lanes, missing signs, odd intersections. Pulling data from OpenStreetMap (a free, open version of Google Maps) means the simulation has to deal with the same messiness a real city planner would.

How the Three Parts Talk to Each Other

The Engine (C++) runs the live simulation, moment to moment. Every so often it asks the Brain (Python) for advice: "given what's happening right now, what should this traffic light do?" The Brain replies almost instantly, and the Engine acts on it without ever pausing the simulation.

ANALOGY — A RACE CAR DRIVER WITH A RADIO TO THE PIT CREW

The driver (the Engine) is doing the actual driving in real time and cannot afford to stop and think too long. Every so often they get a quick radio call from the pit crew (the Brain) with strategic advice — "push harder now," "ease off here" — and act on it instantly without slowing down. Critically: the driver never *waits in silence* for the radio to crackle to life. If the pit crew is slow to respond, the driver keeps driving on the last instruction they got, rather than stalling the car mid-corner.

The Most Important Idea: Fairness, Not Just Speed

It would be easy to build an AI that just minimizes the average commute time — but that AI could "cheat" by making traffic great in wealthy, well-connected areas while quietly ignoring poorer or less-connected neighborhoods. So the AI is specifically taught to care about every zone of the city, not just the average.

ANALOGY — A TEACHER GRADING A WHOLE CLASS, NOT JUST THE TOP STUDENTS

A teacher who only reports the class's average test score could hide the fact that a few students are struggling badly. A good teacher checks how every student — and every group of students — is doing. This project's AI is graded the same way: on how well it treats every part of the city, not just the parts that were already doing fine.

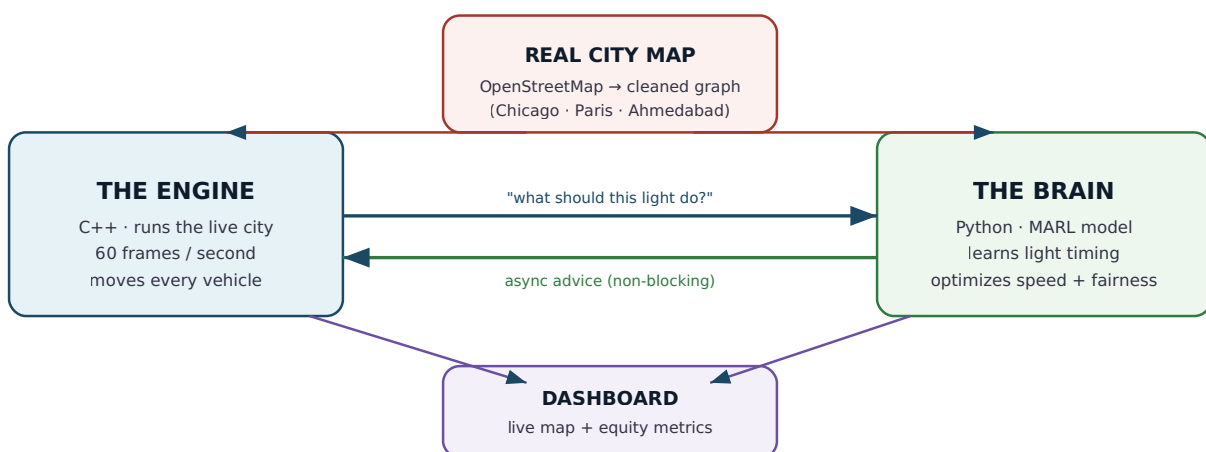


Figure 1 — The three building blocks and how they communicate. The map feeds both the Engine (geometry) and the Dashboard (context); the Engine and Brain talk continuously but never block each other.

System Architecture, End to End

Underneath the analogy is a fairly standard three-tier distributed system, except the "tiers" are a real-time simulation loop, a machine learning inference service, and a geospatial data pipeline — each with its own performance constraints.

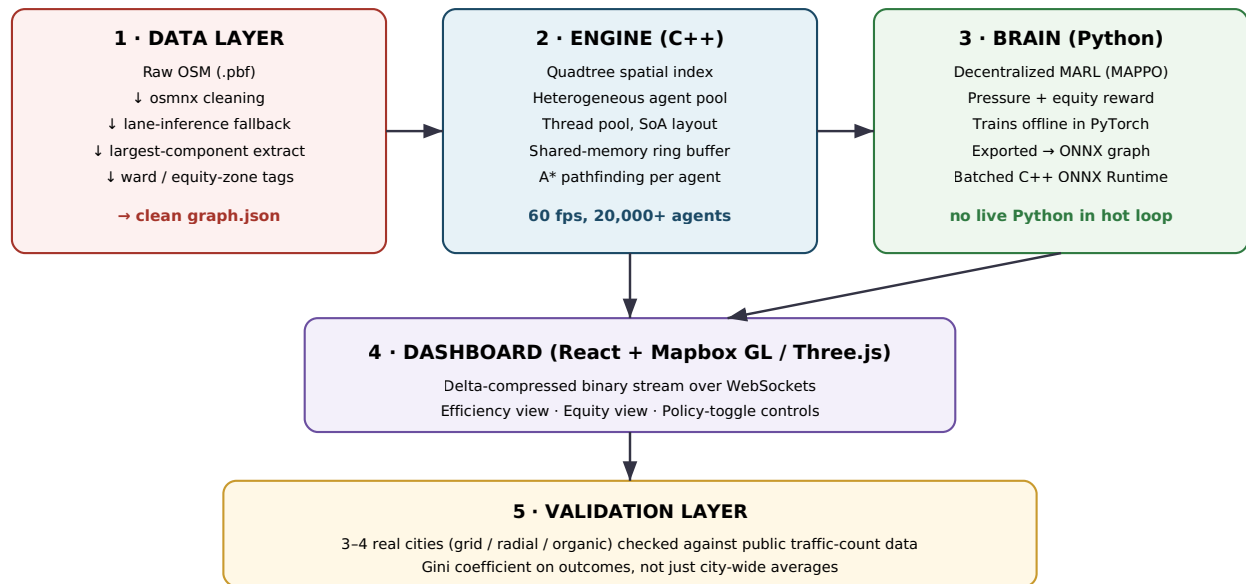


Figure 2 — Five-layer architecture. Each layer is intentionally decoupled behind a documented interface, so any layer can be rebuilt, swapped, or handed to a collaborator without touching the others.

Nine Problems Interviewers Would Raise — Solved

The first version of this idea was reviewed as if by three separate interviewers — an SDE lead, an ML engineer, and a civil services panelist. Between them they surfaced nine real weaknesses. Below is each problem, followed by the concrete engineering fix.

A. Architecture & Systems Engineering (SDE)

PROBLEM 1 — THE SYNCHRONOUS LOOP BOTTLENECK

If the C++ engine makes a blocking network call to the Python brain every frame, serialization and round-trip latency blow past the 16.6 ms budget needed for 60 fps, and the simulation stutters or freezes.

SOLUTION

Decouple the two loops entirely. The C++ engine writes state into a `lock-free shared-memory ring buffer` (or a local gRPC async stream) and immediately continues simulating using the *last known* signal decision — it never waits. A separate low-priority thread reads the Brain's replies whenever they arrive and swaps them in atomically. This is the "radio to the pit crew" from Figure 1: the driver never stops driving to wait for a reply.

PROBLEM 2 — RAW OPENSTREETMAP STRUCTURAL FAILURES

Raw OSM data has overlapping nodes, fragmented geometry, and disconnected sub-graphs. Standard graph traversal (A*, Dijkstra) will crash or silently produce broken routes.

SOLUTION

A dedicated Python cleaning stage runs before anything touches C++: (1) simplify the raw graph with `osmnx`, merging near-duplicate nodes; (2) keep only the *largest strongly-connected component*, discarding orphaned fragments; (3) run a rule-based lane-inference fallback chain — explicit `lanes` tag → road-type lookup table → road-width ÷ standard lane width → classification default — with every inferred edge tagged with a confidence score, so gaps are documented rather than silently guessed.

PROBLEM 3 — THE NETWORK BANDWIDTH CHOKE POINT

Streaming thousands of agent positions as JSON, 60 times a second, floods the WebSocket connection and lags the dashboard.

SOLUTION

Replace JSON with a compact binary format (FlatBuffers, or a hand-rolled packed struct). Send *deltas* only — agents that haven't materially moved or turned are skipped that frame — and apply simple level-of-detail culling so off-screen or zoomed-out agents update less frequently. This cuts payload size by roughly an order of magnitude versus naive JSON streaming.

B. Algorithmic & Deep Learning Problems (ML)

PROBLEM 4 — STATE-ACTION SPACE EXPLOSION

A single model trying to learn optimal signal timing for an entire city at once faces a combinatorial state-action space and will thrash without converging.

SOLUTION

Use a *decentralized* multi-agent design: every intersection is its own lightweight RL agent whose state is only its local queues and its immediate neighbors' pressure — not the whole city. This is the standard fix in traffic-signal MARL literature and keeps each agent's problem small and learnable, while system-wide behavior still emerges from many agents interacting.

PROBLEM 5 — PYTHON'S GIL CHOKING HIGH-FREQUENCY INFERENCE

Running live deep-learning inference for hundreds of intersections through a single-threaded Python process cannot keep up with a high-performance C++ engine.

SOLUTION

Python is only used for *training*, offline. The trained policy is exported to *ONNX* and executed inside C++ itself via the ONNX Runtime C++ API, with requests from many intersections batched into a single tensor call per tick. No live Python process sits in the simulation's hot path at all — this removes the GIL bottleneck by removing Python from the loop entirely.

PROBLEM 6 — THE CREDIT-ASSIGNMENT DILEMMA

A light that clears its own queue by dumping cars onto the next intersection looks locally successful but is a citywide failure — plain "reduce my own wait time" rewards actively encourage this.

SOLUTION

Switch the local reward term from raw queue length to *pressure* — the difference between upstream and downstream queue length. An agent can no longer improve its score by shoving congestion downstream, because pressure explicitly measures that. Full mechanics in Part 5.

C. Data Realism & Public Policy Problems (Civil Services)

PROBLEM 7 — THE HETEROGENEOUS TRAFFIC REALITY GAP

Standard simulations assume uniform vehicles obeying lanes cleanly. Indian metropolitan traffic — autos, two-wheelers, pedestrians sharing unstructured space — breaks that assumption completely.

SOLUTION

Agents are typed, not uniform: each vehicle class (car, bus, auto-rickshaw, two-wheeler, informal shuttle, pedestrian) carries its own speed profile, lane-discipline parameter, and overtaking behavior. A single tunable `chaos coefficient` globally scales how strictly agents obey lane boundaries and right-of-way, so the same engine can represent a highly disciplined Chicago grid and a mixed-traffic Ahmedabad intersection just by changing one parameter — and that parameter itself becomes something worth reporting per city.

PROBLEM 8 — THE SOCIOECONOMIC DEMOGRAPHICS BLIND SPOT

OSM only contains physical road geometry — it has no notion of which neighborhoods are wealthy or underserved, so "fairness" cannot be computed from map data alone.

SOLUTION

Overlay each ward or zone with a proxy signal — building density and footprint size from OSM's own building tags where formal census data isn't available, or real ward-level income/census data where it is. Every core metric (wait time, transit access) is then reported *per zone*, plus a single summary `Gini coefficient` across zones — a standard economics tool that turns "is this fair?" into a number instead of a guess.

PROBLEM 9 — THE REAL-WORLD DATA DEFICIT

A simulation seeded purely from a physical road map, with no real traffic volumes, produces a "utopian" city that doesn't match actual congestion patterns.

SOLUTION

Calibrate against whatever real municipal or public traffic-count data is available for each validation city (public sensor counts, published transit ridership, open congestion indices) to seed origin–destination demand, and be explicit about confidence where such data is sparse — the same documented-uncertainty principle used for the OSM lane-inference fallback in Problem 2.

The Reward Function: Credit Assignment + Fairness, Together

This is the single most important piece of engineering in the whole project, because it's where "looks impressive" and "is actually defensible" diverge. The final reward combines a mechanism that fixes the credit-assignment problem with an objective that actually reflects the equity goal — not two separate patches, one design.

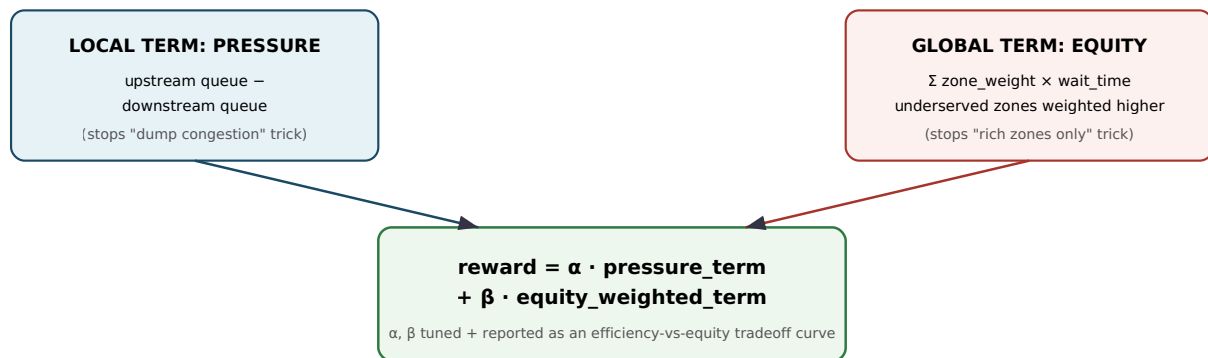


Figure 3 — Reward composition: a local mechanism that eliminates the "shove the jam downstream" exploit, combined with a global term that explicitly favors historically underserved zones over a flat citywide average.

Why this exact combination, and not the alternatives

Approach	What it fixes	Tradeoff
Raw queue length reward	Nothing — this is the naive baseline that fails	Causes the exact downstream-dumping failure mode
Difference rewards (counterfactual)	True per-agent marginal contribution	Mathematically elegant but unstable to train solo, on a real timeline
Pressure-based (chosen)	Downstream dumping, with much simpler implementation	Slightly less precise than counterfactual credit, but well-cited and stable
Flat citywide average objective	Nothing for fairness — a purely efficient agent can starve poor zones	Optimizes the wrong number
Equity-weighted global term (chosen)	Directly penalizes neglecting underserved zones	Requires zone-level demographic proxies (Problem 8's fix)

The tunable weight β is not hidden — it is reported alongside a plotted tradeoff curve (efficiency vs. equity as β varies). That single chart is one of the strongest artifacts in the whole project: it shows,

visually, that a deliberate tradeoff was made and measured, rather than one number being blindly maximized.

The Real-City Data Pipeline

Three structurally different cities are used so that "this generalizes" is a demonstrated claim, not an assumed one: a grid city (Chicago), a radial/ring city (Paris), and an organic mixed-traffic city (Ahmedabad) — with a dense two-wheeler-heavy Southeast Asian city as an optional fourth stretch case.

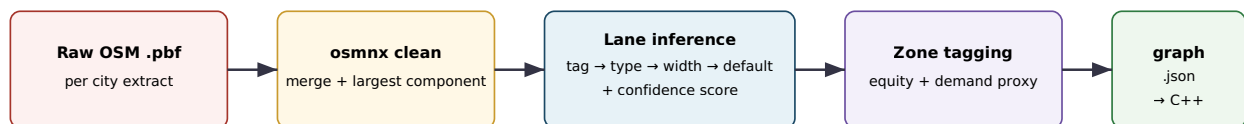


Figure 4 — Pipeline turning messy public map data into a clean, confidence-scored graph the C++ engine can safely traverse.

Validation strategy

- **Chicago (grid):** best OSM data quality and public traffic indices — used first to isolate "is the simulation engine correct" from "is the OSM inference correct."
- **Paris (radial/ring):** genuinely different pathfinding and congestion dynamics from a grid, still well-mapped.
- **Ahmedabad (organic/mixed):** home turf, where the chaos-coefficient and equity-zone work are stress-tested against lived experience.
- **Optional — a dense Southeast Asian city:** heavy two-wheeler dominance, a distinct traffic profile from both the above.

PART 7

Solutions Summary

#	Problem	Fix, in one line
1	Synchronous loop bottleneck	Async shared-memory hand-off; engine never blocks on the Brain
2	Broken raw OSM geometry	Clean, simplify, keep largest component, confidence-scored lane inference
3	Network bandwidth choke	Binary delta-compressed streaming instead of raw JSON
4	State-action space explosion	Decentralized per-intersection MARL agents
5	Python GIL bottleneck	Train in Python, infer via batched ONNX Runtime inside C++
6	Credit-assignment dilemma	Pressure-based local reward term
7	Heterogeneous traffic gap	Typed agents + a single tunable chaos coefficient
8	Socioeconomic blind spot	Zone-level equity tagging + Gini coefficient reporting
9	Real-world data deficit	Calibrate demand against real public traffic-count data, per city

PART 8

The 30-Second Version, If Someone Asks

SAY THIS OUT LOUD

"I'm building a working, fake copy of a real city's traffic using real map data. A fast C++ engine moves thousands of vehicles around smoothly, like a video game. A Python AI watches the traffic and slowly learns the best way to control the lights, through trial and error — not fixed rules. And instead of just making traffic faster on average, the AI is specifically taught to be fair, making sure every part of the city benefits, not just the wealthiest areas. Every rough edge — from messy map data to Python being too slow for real-time use — has its own engineered fix, and it's designed to work for any city in the world, not just one."